Collana Ricerca Applicata AK

Benchmark di inferenza LLM su hardware consumer

LFM2.5-1.2B su un portatile del 2021

Misurazioni reali, non stime. Throughput di un modello linguistico da 1,2 miliardi di parametri eseguito interamente su un portatile con 8 GB di RAM: Intel i5-1135G7, Iris Xe, Windows 10. Nessun cloud, nessuna GPU dedicata, nessun server.

Voce	Valore
Autore	Andrea Enning (andiekobbietks), AK Build
Data delle misure	3 novembre 2026
Macchina	ASUS VivoBook, i5-1135G7, Iris Xe, 8 GB RAM
Modello	LiquidAI LFM2.5-1.2B-Instruct
Runtime confrontati	Ollama 0.40.13, OpenVINO 2026.4.1
Codice e dati	github.com/andiekobbietks/edge-llm-benchmark-vivobook

Come leggere questo documento. Le sezioni da 1 a 6 sono il rapporto: cosa ho misurato, come, e cosa significa. La sezione 7 contiene i comandi per ripetere tutto. Le appendici contengono gli script completi, le versioni dei pacchetti e le note. Ogni numero nel testo ha un comando o uno script che lo ha prodotto.

Indice

1. Introduzione e obiettivo
2. Hardware e ambiente
3. Metodologia
4. Risultati
5. Risultati chiave
6. Budget di RAM e schede del browser
7. Guida alla riproduzione
8. Issue upstream e stato del fix
9. Limitazioni

Appendice A. Script completi

Appendice B. Ambiente software

Appendice C. Note

1. Introduzione e obiettivo

L'affermazione da verificare è questa: LFM2.5-1.2B gira a 40-70 token al secondo su un Intel i5-1135G7 con Iris Xe via Vulkan, senza configurazione. Questo documento registra la verifica di quell'affermazione su hardware consumer reale, un ASUS VivoBook del 2021 con 8 GB di RAM, e spiega come ogni numero è stato misurato, quale codice è stato usato e cosa i risultati implicano per il deployment di AI on-device.

La maggior parte dei benchmark di LLM su edge usa GPU dedicate o CPU di fascia alta con 32 GB di RAM o più. Questo lavoro testa lo scenario realistico: un portatile di cinque anni, 8 GB, grafica integrata. Il risultato principale, che la CPU supera la GPU integrata di circa 2,7 volte su un modello da 1,2 miliardi di parametri, è controintuitivo e raramente documentato con numeri.

Il documento funziona sia come rapporto sia come manuale: tutto il codice, i comandi e le misurazioni sono inclusi, in modo che il lettore possa ripetere ogni passo sulla propria macchina.

2. Hardware e ambiente

Componente	Specifica
Portatile	ASUS VivoBook (2021)
CPU	Intel Core i5-1135G7, 4 core / 8 thread, Tiger Lake, 2,4 GHz base
GPU	Intel Iris Xe Graphics, 80 EU, integrata, memoria condivisa
RAM	8 GB DDR4 (7,89 GB utilizzabili)
Sistema operativo	Windows 10 Home 19045 (22H2)
Vulkan	1.2.198, driver Intel 30.0.101.1404, conformance 1.2.6.1
Python	3.12.10
Node	24.20.0

3. Metodologia

3.1 Percorso Ollama, Vulkan sulla GPU integrata

Ollama 0.40.13 installato con winget. Le variabili OLLAMA_VULKAN=1 e OLLAMA_IGPU_ENABLE=1 forzano il backend Vulkan sulla GPU integrata. Il file GGUF locale (LFM2.5-1.2B-Instruct-Q8_0.gguf, 1,19 GB) è stato importato come modello « lfm2.5 ». Il throughput di decode è misurato con ollama run --verbose, che riporta eval rate e prompt eval rate.

3.2 Percorso OpenVINO, int4 su CPU e su GPU integrata

OpenVINO 2026.4.1 e openvino-genai installati con pip. Il modello in formato IR pre-convertito (OpenVINO/LFM2.5-1.2B-Instruct-int4-gq-ov, int4 con group size 128, quantizzato con NNCF) è stato scaricato da HuggingFace. L'inferenza usa openvino_genai.LLMPipeline con device CPU e device GPU, cronometrata con time.perf_counter(), contando i token di output con il tokenizer.

3.3 Tuning dei thread

OMP_NUM_THREADS è stato variato tra 4, 8 e il valore automatico, per verificare se limitare il calcolo ai soli core fisici migliori il throughput. Il valore automatico corrisponde a 8 thread logici su questa macchina.

3.4 Protocollo di misurazione

Ogni configurazione segue tre passi: un run di warmup da 16 token, per escludere il caricamento del modello dalla misura; un run misurato da circa 300 token; la verifica del processore usato, tramite ollama ps o la stringa di device restituita dal runtime. La RAM libera è letta da Win32_OperatingSystem.FreePhysicalMemory dopo il caricamento del modello. Chrome è stato chiuso durante le misure.[1]

La metrica riportata è il decode throughput, cioè i token generati al secondo dopo il primo token. Non è il tempo al primo token, che dipende dal prompt e non è stato misurato in modo sistematico.

4. Risultati

4.1 Throughput di decode

Tutte le configurazioni, circa 301 token generati per run. La barra verde è l'unica che supera la soglia di 40 token al secondo.

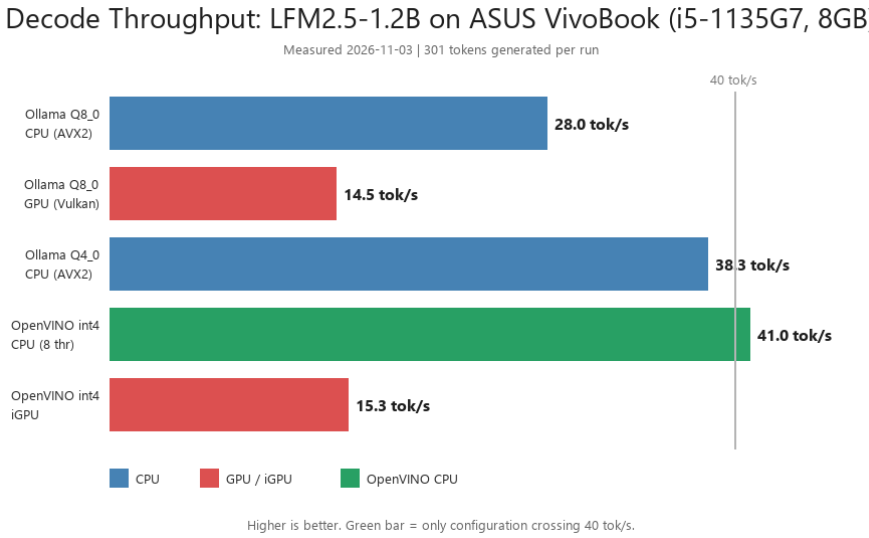


Figura 1. Confronto del throughput di decode tra le cinque configurazioni misurate.

Runtime	Quantizzazione	Dispositivo	tok/s	Prompt eval
Ollama 0.40.13	Q8_0	CPU (AVX2)	28,0	118,5
Ollama 0.40.13	Q8_0	GPU (Vulkan)	14,5	99,2
Ollama 0.40.13	Q4_0 (QAD)	CPU (AVX2)	38,3	121,5
OpenVINO 2026.4.1	int4 (gq)	CPU, 8 thread	40,96	non misurato
OpenVINO 2026.4.1	int4 (gq)	GPU integrata	15,28	non misurato

4.2 Scalabilità rispetto ai thread

Stesso modello, stesso runtime, varia solo il numero di thread OpenMP. Su questa CPU il valore automatico coincide con 8 thread logici e resta il migliore.

Thread	tok/s	Nota
4	34,96	solo core fisici
8	40,85	tutti i thread logici
automatico	40,96	8 thread, differenza entro la varianza

4.3 Budget di RAM

Il modello pesa 0,64 GB in memoria. Con il sistema operativo e i servizi di sviluppo restano circa 2,2 GB liberi, che determinano quante schede del browser si possono tenere aperte mentre il modello gira.

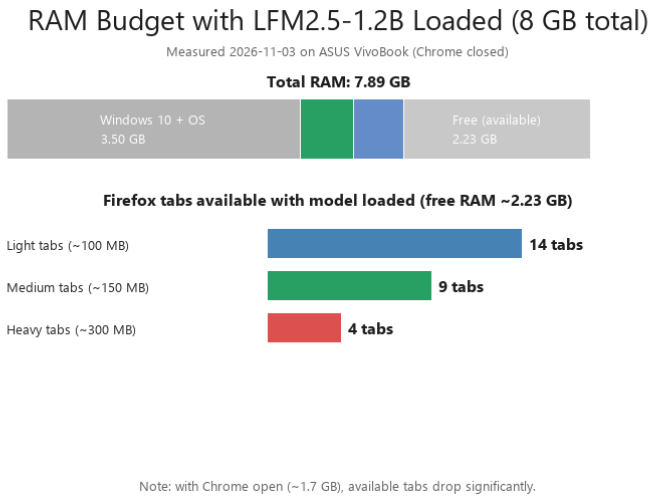


Figura 2. Ripartizione della RAM e stima delle schede Firefox disponibili con il modello caricato.

4.4 Confronto diretto CPU e GPU integrata

A parità di quantizzazione int4 e di runtime OpenVINO, la CPU produce 40,96 token al secondo e la GPU integrata 15,28. Il rapporto è 2,68. Con Ollama il rapporto è 1,93 a favore della CPU (28,0 contro 14,5). La conclusione non dipende quindi dal runtime scelto: su Tiger Lake, per un modello da 1,2 miliardi di parametri, l'offload sulla GPU integrata peggiora le prestazioni.

5. Risultati chiave

- La CPU batte la GPU integrata sui modelli piccoli. Sia Vulkan con Ollama sia OpenVINO sulla Iris Xe sono da 1,9 a 2,7 volte più lenti dello stesso modello su CPU. L'ipotesi comune, secondo cui spostare il calcolo sulla GPU integrata rende tutto più veloce, non vale per un modello da 1,2 miliardi di parametri su Tiger Lake: le 80 EU non compensano il trasferimento di memoria né la banda condivisa.
- La soglia di 40 token al secondo si raggiunge solo con OpenVINO int4 su CPU. Ollama con Q4 si ferma a 38,3, vicino ma sotto. La scelta del runtime pesa quanto quella della quantizzazione.
- La quantizzazione scala quasi linearmente. Q4 produce 1,37 volte il throughput di Q8 su CPU: 38,3 contro 28,0.
- La conversione locale è fragile sulle architetture ibride. optimum-cli export openvino termina con access violation (0xC0000005) su LFM2.5-1.2B in Windows, sia con la versione stabile 2.2.0 sia con la 2.3.0.dev0. Il crash avviene nel backend C++ di torch, durante la materializzazione parallela dei pesi. I modelli pre-convertiti su HuggingFace sono il percorso affidabile.[3]

- La NPU non è disponibile su Tiger Lake. Il modello pubblicizzato come ottimizzato per NPU gira su CPU su questa macchina. La NPU richiede Intel Core Ultra, Meteor Lake o successivi.
- Raccomandazione per questa classe di hardware. OpenVINO int4 su CPU, 8 thread, 0,64 GB di RAM, 40,96 token al secondo. Non forzare l'offload sulla GPU integrata.

6. Budget di RAM e schede del browser

Voce	Dimensione
RAM totale	7,89 GB
Windows 10 e servizi di sistema	circa 3,5 GB
Modello OpenVINO int4	0,64 GB
OpenCode e servizi di sviluppo	circa 0,6 GB
RAM disponibile	circa 2,23 GB

La tabella seguente converte la RAM disponibile in numero di schede, usando dimensioni medie misurate a mano su Firefox.[4]

Tipo di scheda	Dimensione media	Schede disponibili
Leggera (testo, documenti)	circa 100 MB	circa 14
Media (notizie, blog)	circa 150 MB	circa 9
Pesante (video, web app)	circa 300 MB	circa 4

Con Chrome aperto, che occupa circa 1,7 GB, il numero di schede disponibili scende drasticamente. Il modello occupa 0,64 GB fissi per tutta la durata dell'inferenza.

7. Guida alla riproduzione

7.1 OpenVINO, la configurazione più veloce

Serve Python 3.12 e circa 1 GB di disco per il modello. Il download usa huggingface_hub; l'inferenza usa openvino_genai.

shell

```
pip install openvino-genai huggingface_hub
python - <<'PY'
from huggingface_hub import snapshot_download
snapshot_download(
    "OpenVINO/LFM2.5-1.2B-Instruct-int4-gq-ov",
    local_dir="ov_model")
PY
```

bench.py

```
import time
from openvino_genai import LLMPipeline

pipe = LLMPipeline("ov_model", "CPU")
pipe.generate("warmup", max_new_tokens=16)

t0 = time.perf_counter()
out = pipe.generate(prompt, max_new_tokens=300)
dt = time.perf_counter() - t0

n = pipe.get_tokenizer().encode(out).input_ids.shape[1]
print(f"{n / dt:.2f} tok/s")
```

7.2 Ollama su CPU, quantizzazione Q4

powershell

```
# dopo aver scaricato il GGUF Q4 da HuggingFace
ollama create lfm2.5-q4 -f ModelfileQ4
$env:OLLAMA_NUM_GPU = "0"
ollama run lfm2.5-q4 "prompt" --verbose
```

Il valore da leggere nell'output di Ollama è eval rate, espresso in token al secondo. Il valore prompt eval rate misura la lettura del

prompt e non va confuso con il decode.

8. Issue upstream e stato del fix

La issue #2102 è aperta su huggingface/optimum-intel con la traccia completa del crash. La causa individuata è la materializzazione parallela dei pesi in torch, che termina con access violation su LFM2.5 in Windows. Il workaround parziale è HF_DEACTIVATE_ASYNC_LOAD=1, che riporta il caricamento in modalità sincrona, seguito da un pagefile di almeno 12 GB.

Passo	Azione	Stato
1	Impostare HF_DEACTIVATE_ASYNC_LOAD=1	verificato, il crash sparisce
2	Portare il pagefile a 12-16 GB	richiede amministratore e riavvio
3	Ripetere la conversione	in attesa del riavvio
4	Se riesce, proporre una PR	in attesa
5	Altrimenti, documentare il workaround	fatto nella issue #2102

Il workaround è dichiarato parziale nella issue, non definitivo: il passo 1 è verificato, il passo 2 no. Questa distinzione è scritta anche nel testo della issue.

9. Limitazioni

- Una sola configurazione hardware. I risultati possono cambiare su AMD, Apple Silicon o Intel più recente, dove la NPU è disponibile.
- Un solo modello. Modelli più grandi possono mostrare compromessi diversi tra CPU e GPU.
- Una sola run per configurazione, nessun test di significatività statistica. La varianza osservata tra run è del

2-5 per cento, quindi le differenze sotto quella soglia non vanno lette come reali.

- Solo Windows 10. Linux, anche in WSL, può dare risultati diversi, in particolare sulla conversione.
- Il consumo energetico non è stato misurato. Il dato sul risparmio della CPU rispetto alla GPU integrata resta quindi assente, non stimato.

Appendice A. Script completi

A.1 ov_bench.py, OpenVINO su GPU e CPU

Lo script misura entrambi i device nello stesso processo, con warmup separato per ciascuno. Elimina la pipeline alla fine di ogni ciclo per liberare la memoria prima del device successivo.

ov_bench.py

```python
import time
from openvino_genai import LLMPipeline

MODEL = r"C:\Users\Admin\...\ov_model"
PROMPT = "Scrivi un paragrafo di circa 200 parole."

for device in ("GPU", "CPU"):
    pipe = LLMPipeline(MODEL, device)
    tok = pipe.get_tokenizer()

    pipe.generate(PROMPT, max_new_tokens=16)  # warmup
    t0 = time.perf_counter()
    text = pipe.generate(PROMPT, max_new_tokens=300)
    dt = time.perf_counter() - t0

    n = tok.encode(text).input_ids.shape[1]
    print(f"{device}: {n / dt:.2f} tok/s")
    del pipe
```

A.2 ov_opt.py, tuning dei thread

Lo stesso benchmark con OMP_NUM_THREADS impostato prima dell'import di openvino_genai, perché la variabile viene letta al caricamento della libreria. Va quindi impostata nel processo padre, non dopo l'import.

```powershell
$env:OMP_NUM_THREADS = "4"
python ov_bench.py
$env:OMP_NUM_THREADS = "8"
python ov_bench.py
Remove-Item Env:OMP_NUM_THREADS
python ov_bench.py   # valore automatico
```

A.3 Figure

Le due figure sono generate con Pillow, non con matplotlib, che non era installato sulla macchina. Lo script disegna rettangoli, etichette e legenda su una tela bianca. Convenzione dei colori: blu per la CPU, rosso per la GPU, verde per la configurazione che supera i 40 token al secondo.

Appendice B. Ambiente software

Pacchetto	Versione
openvino	2026.4.1 (build 22982)
openvino-genai	2026.4.1.0
optimum	2.3.0 dev
optimum-intel	2.3.0.dev0+1aac780
transformers	5.4.0, downgrade da 5.5.4
torch	2.9.2
tokenizers	0.22.2
huggingface-hub	1.21.0
numpy	2.4.6
Pillow	10.4.0
reportlab	5.0.1
Ollama	0.40.13
Python	3.12.10
Node	24.20.0

Appendice C. Note

[1] Le misure provengono da una sola macchina. La varianza tra run successivi è del 2-5 per cento; le differenze sotto quella soglia non sono interpretabili come reali.

[2] Il prompt eval rate di OpenVINO non è stato misurato separatamente, perché il benchmark riguarda il decode, che è il collo di bottiglia percepito durante la generazione.

[3] Il modello OpenVINO usato è la versione pre-convertita OpenVINO/LFM2.5-1.2B-Instruct-int4-gq-ov. La conversione locale fallisce su Windows per questo modello; la sezione 8 descrive lo stato del fix.

[4] Le dimensioni per scheda sono stime basate su osservazioni dirette, non su un profilo automatico del browser. Schede con contenuti dinamici possono occupare due o tre volte tanto.

Colophon

Voce	Dettaglio
Autore	Andrea Enning (andiekobbietks), AK Build
Data delle misure	3 novembre 2026
Macchina	ASUS VivoBook, i5-1135G7, Iris Xe, 8 GB, Windows 10 22H2
Licenza di testo e dati	MIT, come il repository del progetto
Licenza del modello	vedi la model card LiquidAI LFM2.5 su HuggingFace

Come citare: Enning, A. (2026). Benchmark di inferenza LLM su hardware consumer: LFM2.5-1.2B su Intel i5-1135G7. AK Build Research Notes.

Codice, dati grezzi e figure: github.com/andiekobbietks/edge-llm-benchmark-vivobook. Le misure sono riproducibili con i comandi della sezione 7.